2算法的基本逻辑能力（下）

18.循环

人vs计算机



- 人的方式：眼睛一看就知道了
  - 352 —> 3位！
- 计算机的方式：判断数的范围来决定它的位数
  - 352∈[100,999] —> 3位
  - 人不擅长，因为人对数字的计算能力比文字弱

程序实现

```
int x;
int n =1;

scanf("%d", &x);

if ( x > 999 ) {
    n = 4;
} else if ( x > 99 ) {
    n = 3;
} else if ( x > 9 ) {
    n = 2;
}

printf("%d\n", n);
```

- 因为题目明确了4位数及以下的正整数，所以可以简化一些判断
- 因为从高处往下判断，所以不需要判断上限了
  - 反过来不行
- 问题：任意范围的正整数怎么办？

```c
int x;
int n =0;

scanf("%d", &x);

if ( x > 999 ) {
    n = 4;
} else if ( x > 99 ) {
    n = 3;
} else if ( x > 9 ) {
    n = 2;
} else if ( x > 0 ) {
    n = 1;
}

printf("%d\n", n);
```

```c
int x;
int n =0;

scanf("%d", &x);

if ( x > 9999 ) {
    n = 5;
} else if ( x > 999 ) {
    n = 4;
} else if ( x > 99 ) {
    n = 3;
} else if ( x > 9 ) {
    n = 2;
} else if ( x > 0 ) {
    n = 1;
}

printf("%d\n", n);
```

```c
int x
int r

scanf("%d", &x);

if ( x > 99999 ) {
    n = 6;
} else if ( x > 9999 ) {
    n = 5;
} else if ( x > 999 ) {
    n = 4;
} else if ( x > 99 ) {
    n = 3;
} else if ( x > 9 ) {
    n = 2;
} else if ( x > 0 ) {
    n = 1;
}

printf("%d\n", n);
```

你写的不是关系，不是说明，而是步骤。

数数

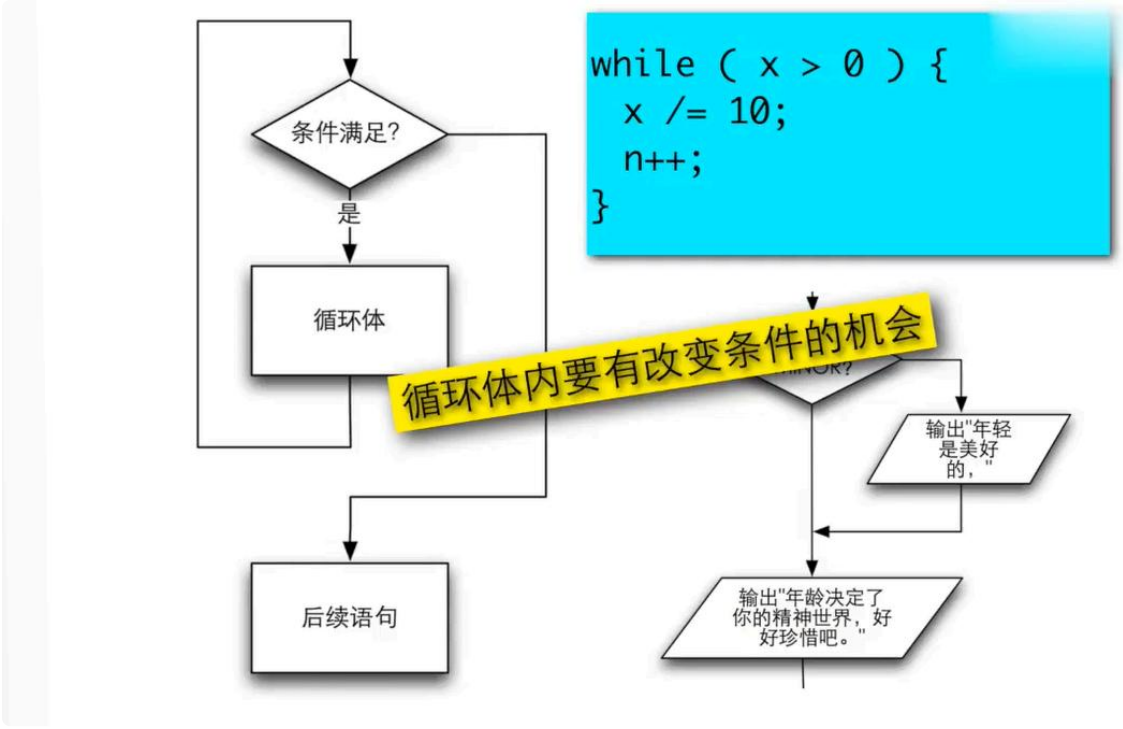


- 12381284326751827361827361267531 7
- 人怎么数？从左往右数，一次划掉一个数字
- 计算机怎么划掉那个数字？

```
4  {
5      int x;
6      int n=0;
7      scanf("%d", &x);
8      n++;
9      x /= 10;
10     while ( x> 0 ) {
11         n++;
12         x /= 10;
13     }
14     printf("%d\n", n);
15     return 0;
16 }
```

注：先别拿太大的数字去尝试

19.WHILE循环



while循环



- 如果我们把while翻译作“当”，那么一个while循环的意思就是：当条件满足时，不断地重复循环体内的语句。

- 循环执行之前判断是否继续循环，所以有可能循环一次也没有被执行；
- 条件成立是循环继续的条件。

再想想



```c
int x;
int n =0;

scanf("%d", &x);

n++;
x /= 10;
while ( x > 0 ) {
    n++;
    x /= 10;
}

printf("%d\n", n);
```

```c
int x;
int n =0;

scanf("%d", &x);

while ( x > 0 ) {
    n++;
    x /= 10;
}

printf("%d\n", n);
```

- 如果没有外面的运算?

看程序运行结果



- 人脑模拟计算机的运行，在纸上列出所有的变量，随着程序的进展不断重新计算变量的值。当程序运行结束时，留在表格最下面的就是程序的最终结果

写纸上可以增强分析能力。而且不是所有的程序都方便直接用计算机算。

验证

- 测试程序常使用边界数据，如有效范围两端的数据、特殊的倍数等
  - 个位数；
  - 10；
  - 0；
  - 负数。

测试这些数据是为了能够全方位地检验你的代码是否正确。

```c
#include <stdio.h>

int main()
{
    int x;
    int n=0;

    scanf("%d", &x);
    if ( x>0 ) {
        while ( x>0 ) {
            n++;
            x /= 10;
        }
    } else {
        n = 1;
    }
    printf("%d\n", n);

    return 0;
}

x   n
5   0
0   1
```

调试



```c
int x;
int n =0;

scanf("%d", &x);

n++;
x /= 10;
while ( x > 0 ) {
    printf("x=%d, n=%d\n", x, n);
    n++;
    x /= 10;
}

printf("%d\n", n);
```

- 在程序适当的地方插入printf来输出变量的内容

loop.c

```c
#include <stdio.h>

int main()
{
    int x;
    int n=0;

//  scanf("%d", &x);
    x = 352;
    while ( x>0 ) {
        n++;
        x /= 10;
        printf("x=%d,n=%d\n", x, n);
    }
    printf("%d\n", n);

    return 0;
}
```

```
=35,n=1
=3,n=2
=0,n=3

Finished in 0.8s]
```

```c
int main()
{
    int x;
    int n=0;

//  scanf("%d", &x);
    x = 0;
    while ( x>0 ) {
        printf("in loop\n");
        n++;
        x /= 10;
        printf("x=%d,n=%d\n", x, n);
    }
    printf("%d\n", n);

    return 0;
}
```

```c
#include <stdio.h>

int main()
{
    int x;
    int n=0;

//  scanf("%d", &x);
    x = 0;
    while ( x>0 ) {
        printf("hr1\n");
        n++;
        x /= 10;
        printf("x=%d,n=%d\n", x, n);
    }
    printf("%d\n", n);

    return 0;
}
```

`hr`用来表明程序到哪里了，这样可以少一点debug。这也是一种常见的调试手段。补充，有的时候，debug不一定开的出来。

注：把代码放到某些练习测试平台，要把debug给注释掉，防止误判。

```c
// 老师代码的思路
n++;
x /= 10;
while (x > 0) {
    n++;
    x /= 10;
}
```

如果输入 `0`：直接先执行外面的代码。`n = 1`，`x = 0 / 10 = 0`。然后进入 `while(0 > 0)`，不成立，跳过。最后输出 `1`。处理了 0的情况。

如果输入 `123`：先执行外面：n=1，x = 123 / 10 = 12。（相当于已经剥掉了个位 3）

进循环第一轮：`n=2`，`x = 1`。

进循环第二轮：`n=3`，`x = 0`。

第三轮 `0 > 0` 不成立，退出。输出 `3`

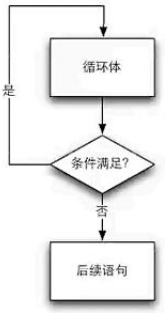
20.DO-WHILR循环

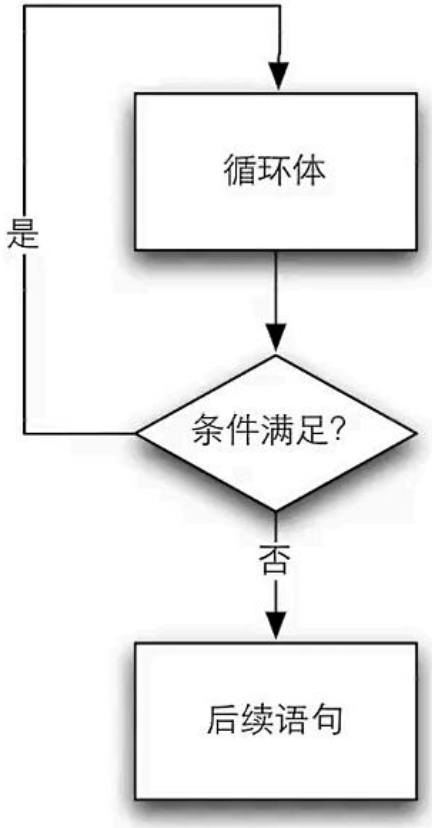
do-while循环

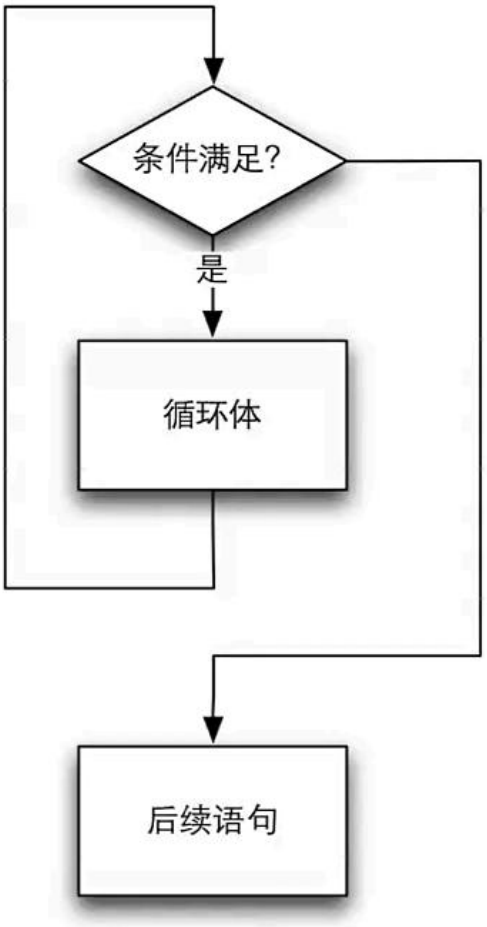
- 在进入循环的时候不做检查，而是在执行完一轮循环体的代码之后，再来检查循环的条件是否满足，如果满足则继续下一轮循环，不满足则结束循环



```
do
{
    <循环体语句>
} while ( <循环条件> );
```







左do-while 右while

两种循环



- do-while循环和while循环很像，区别是在循环体执行结束的时候才来判断条件。也就是说，无论如何，循环都会执行至少一遍，然后再来判断条件。与while循环相同的是，条件满足时执行循环，条件不满足时结束循环。



```
int x;
scanf("%d", &x);
int n = 0;
do
{
  x /= 10;
  n ++;
} while ( x > 0 );
printf("%d", n);
```

小心

21.猜数

猜数游戏



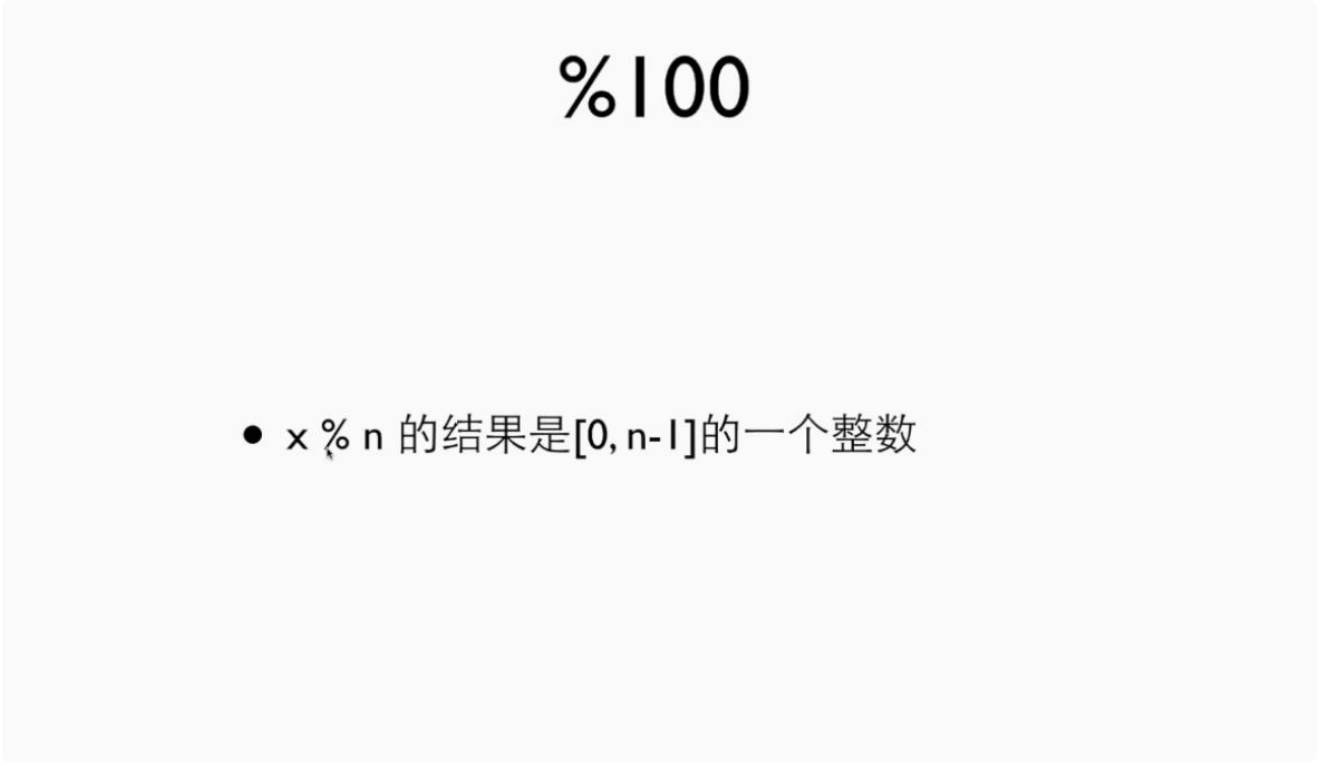
- 让计算机来想一个数，然后让用户来猜，用户每输入一个数，就告诉它是大了还是小了，直到用户猜中为止，最后还要告诉用户它猜了多少次。
- 因为需要不断重复让用户猜，所以需要用到循环
- 在实际写出程序之前，我们可以先用文字描述程序的思路
- 核心重点是循环的条件
  - 人们往往会考虑循环终止的条件



1. 计算机随机想一个数，记在变量number里；
2. 一个负责计次数的变量count初始化为0；
3. 让用户输入一个数字a；
4. count递增（加一）；
5. 判断a和number的大小关系，如果a大，就输出“大”；如果a小就输出“小”；
6. 如果a和number是不相等的（无论大还是小），程序转回到第3步；
7. 否则，程序输出“猜中”和次数，然后结束。

循环条件是a和number不相等

```
srand(time(0));
int number = rand()%100+1;
int count = 0;
int a = 0;
printf("我已经想好了一个1到100之间的数。");
do {
  printf("请猜这个1到100之间数：");
  scanf("%d", &a);
  count ++;
  if ( a > number ) {
    printf("你猜的数大了。");
  } else if ( a < number ) {
    printf("你猜的数小了。");
  }
} while (a != number);
printf("太好了，你用了%d次就猜到了答案。\n", count);
```




```
srand(time(0));
int number = rand()%100+1;
int count = 0;
int a = 0;
printf("我已经想好了一个1到100之间的数。");
do {
  printf("请猜这个1到100之间数：");
  scanf("%d", &a);
  count ++;
  if ( a > number ) {
    printf("你猜的数大了。");
  } else if ( a < number ) {
    printf("你猜的数小了。");
  }
} while (a != number);
printf("太好了，你用了%d次就猜到了答案。\n", count);
```

22.算平均数

算平均数

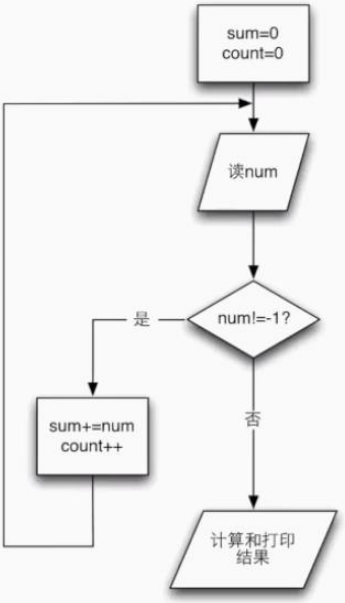
$$sum = \frac{\sum_{i=1}^{n} x_i}{n}$$



- 让用户输入一系列的正整数，最后输入-1表示输入结束，然后程序计算出这些数字的平均数，输出输入的数字的个数和平均数
- 变量->算法->流程图->程序

算法

1. 初始化变量sum和count为0；
2. 读入number；
3. 如果number不是-1，则将number加入sum，并将count加1，回到2；
4. 如果number是-1，则计算和打印出sum / count（注意换成浮点来计算）。



```c
#include <stdio.h>

int main()
{
    int number;
    int sum = 0;
    int count = 0;

    scanf("%d", &number);
    while ( number != -1 ) {
        sum += number;
        count ++;
        scanf("%d", &number);
    }

    printf("%f\n", 1.0*sum/count);

    return 0;
}
```

```c
do {
    scanf("%d", &number);
    if ( number != -1 ) {
        sum += number;
        count ++;
    }
} while ( number != -1 );
```

23.整数逆序

整数的分解

- 一个整数是由1至多位数字组成的，如何分解出整数的各个位上的数字，然后加以计算
  - 对一个整数做%10的操作，就得到它的个位数；
  - 对一个整数做/10的操作，就去掉了它的个位数；
  - 然后再对2的结果做%10，就得到原来数的十位数了；
  - 依此类推。

数的逆序

- 输入一个正整数，输出逆序的数
  - 结尾的0的处理

reverse.c

```c
#include <stdio.h>

int main()
{
    int x;
    // scanf("%d", &x);
    x = 700;
    int digit;
    int ret = 0;

    while ( x> 0 ) {
        digit = x%10;
        printf("%d\n", digit);
        ret = ret*10 + digit;
        printf("x=%d,digit=%d,ret=%d\n", x, digit, ret);
        x /= 10;
    }
    printf("%d", ret);

    return 0;
}
```

```
x=700,digit=0,ret=0
x=70,digit=0,ret=0
x=7,digit=7,ret=7
7[Finished in 0.1s]
```

reverse.c

```c
#include <stdio.h>

int main()
{
    int x;
    // scanf("%d", &x);
    x = 700;
    int digit;
    int ret = 0;

    while ( x> 0 ) {
        digit = x%10;
        printf("%d", digit);
        ret = ret*10 + digit;
        // printf("x=%d,digit=%d,ret=%d\n", x, digit, ret);
        x /= 10;
    }
    // printf("%d", ret);

    return 0;
}
```

```
007[Finished in 0.1s]
```

24FOR循环

阶乘

- n! = 1×2×3×4×...×n
- 写一个程序，让用户输入n，然后计算输出n!
- 变量：
  - 显然读用户的输入需要一个int的n，然后计算的结果需要用一个变量保存，可以是int的factor，在计算中需要有一个变量不断地从1递增到n，那可以是int的i

```
int n;

scanf("%d", &n);
int fact = 1;

int i = 1;
while ( i <= n ) {
    fact *= i;
    i++;
}

printf("%d!=%d\n", n, fact);
```

```c
int n;

scanf("%d", &n);
int fact = 1;

int i = 1;
for ( i=1; i<=n; i++ ) {
    fact *= i;
}

printf("%d!=%d\n", n, fact);
```

for循环

for循环像一个计数循环：设定一个计数器，初始化它，然后在计数器到达某值之前，重复执行循环体，而每执行一轮循环，计数器值以一定步进进行调整，比如加1或者减1

```c
for ( i=0; i<5; i=i+1 ) {
    printf("%d", i);
}
```

for = 对于

- for (count=10; count>0; count--)
- 就读成：“对于一开始的count=10，当count>0时，重复做循环体，每一轮循环在做完循环体内语句后，使得count--。”

小套路

- 做求和的程序时，记录结果的变量应该初始化为0，而做求积的变量时，记录结果的变量应该初始化为1

- 循环控制变量i只在循环里被使用了，环外面它没有任何用处。因此，我们可以把变量i的定义写到for语句里面去

```
int n;

scanf("%d", &n);
int fact = 1;
    C99 ONLY!
for ( int i=1; i<=n; i++ ) {
    fact *= i;
}

printf("%d!=%d\n", n, fact);
```

try

- 1×1还是1，所以程序的循环不需要从1开始，那么改成从多少开始合适呢？这样修改之后，程序对所有的n都正确吗？这样的改动有价值吗？

```c
#include <stdio.h>

int main()
{
    int n;

    scanf("%d", &n);
    int fact = 1;

    int i = 1;
    for ( i=2; i<=n; i++ ) {
        fact *= i;
    }

    printf("%d!=%d\n", n, fact);

    return 0;
}
```

try

- 1×1还是1，所以程序的循环不需要从1开始，那么改成从多少开始合适呢？这样修改之后，程序对所有的n都正确吗？这样的改动有价值吗？
- 除了可以从1乘到n来计算n!，还可以从n乘到1来计算吧？试试把循环换个方向来计算n。这时候，还需要循环控制变量i吗？

```c
#include <stdio.h>

int main()
{
    int n;

    scanf("%d", &n);
    int fact = 1;

    int i = n;
    // for ( i=1; i<=n; i++ ) {
    //  fact *= i;
    // }
    for ( ; n>1; n-- ) {
        fact *= n;
    }
    printf("%d!=%d\n", i, fact);

    return 0;
}
```

25循环的计算和选择

循环次数

- for (i=0; i<n; i++)
- 则循环的次数是n，而循环结束以后，i的值是n。循环的控制变量i，是选择从0开始还是从I开始，是判断i<n还是判断i<=n，对循环的次数，循环结束后变量的值都有影响

```c
#include <stdio.h>

int main()
{
    int i;

    for ( i=0; i<5; i++ )  {
        printf("i=%d ", i);
    }
    printf("\n最后i=%d\n", i);

    return 0;
}
```

```
=0 i=1 i=2 i=3 i=4
最后i=5
```

```c
int n;

scanf("%d", &n);
int fact = 1;

int i = 1;
while ( i <= n ) {
    fact *= i;
    i++;
}

printf("%d!=%d\n", n, fact);
```

```c
int n;

scanf("%d", &n);
int fact = 1;

int i = 1;
for ( i=1; i<=n; i++ ) {
    fact *= i;
}

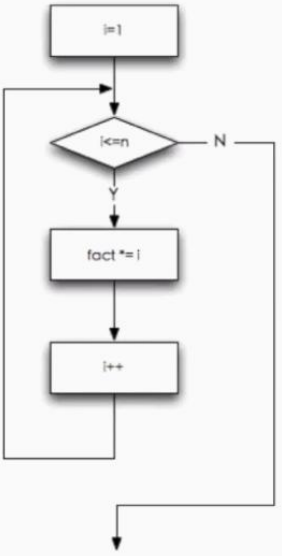
printf("%d!=%d\n", n, fact);
```

for == while

```c
for ( int i=1; i<=n; i++ ) {
    fact *= i;
}
```

=

```c
int i=1;
while (i<=n) {
    fact *=i;
    i++;
}
```



for循环

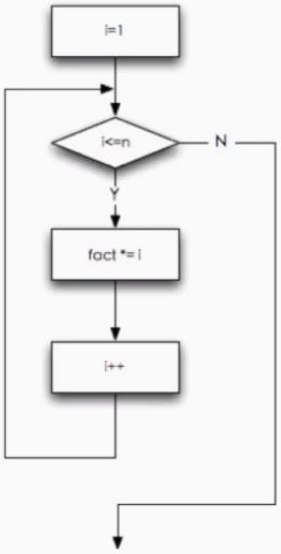
for (初始动作;条件; 每轮的动作) {

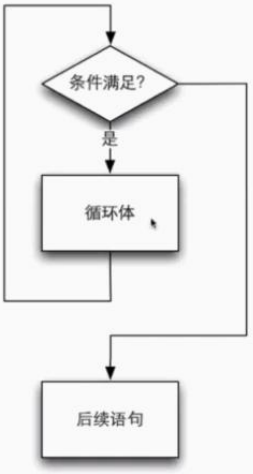
}

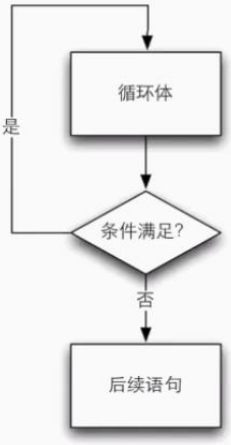
- for中的每一个表达式都是可以省略的

for (; 条件;) == while (条件)

三种循环









26循环的控制

素数：只能被1和自己整除的数，不包括1。如2，3，5，7，11，13，17，19...

```c
#include <stdio.h>

int main()
{
    int x;

    scanf("%d", &x);

    int i;
    int isPrime = 1;    //  x是素数
    for ( i=2; i<x; i++ ) {
        if ( x % i == 0 ) {
            isPrime = 0;
        }
    }
    if ( isPrime == 1 ) {
        printf("是素数\n");
    } else {
        printf("不是素数\n");
    }
    return 0;
}
```

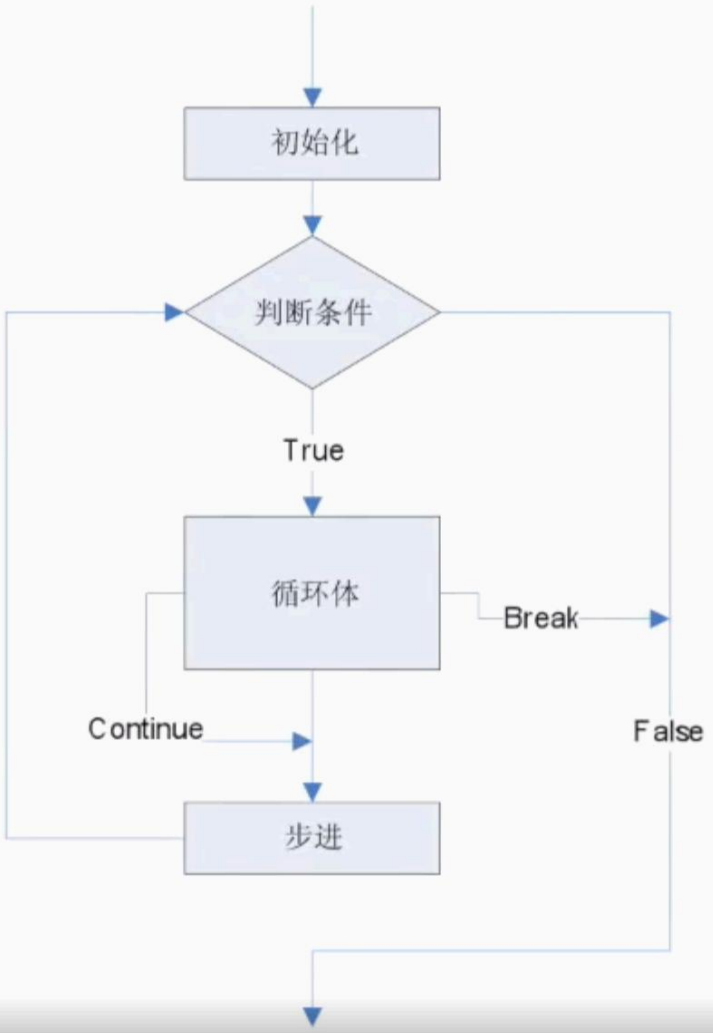
通过无法证伪，来证明它是真的。

```
6
7     // scanf("%d", &x);
8     x = 6;
9     int i;
10    int isPrime = 1;     //  x是素数?
11    for ( i=2; i<x; i++ ) {
12        if ( x % i == 0 ) {
13            isPrime = 0;
14            break;
15        }
16    }
17    if ( isPrime == 1 ) {
18        printf("是素数\n");
```

break vs continue

- break: 跳出循环
- continue: 跳过循环这一轮剩下的语句进入下一轮

break vs continue



63

```c
#include <stdio.h>

int main()
{
    int x;

    scanf("%d", &x);

    int i;
    // int isPrime = 1; //  x是素数
    for ( i=2; i<x; i++ ) {
        if ( x % i == 0 ) {
            // isPrime = 0;
            break;
        }
    }
    // if ( isPrime == 1 ) {
    if ( i< x )
        printf("不是素数\n");
    } else {
        printf("是素数\n");
    }
    return 0;
}
```

这是改进版。但是这段代码写的不好。

因为它在严谨性（边界情况）、运行效率（时间复杂度）和逻辑表达（可读性）上存在三个明显的硬伤。

硬伤一：致命的边界问题（输入 1 会报错）
这段代码最大的漏洞是完全没有处理 1、0 或负数。
推演一下：如果用户输入 x = 1。

for (i = 2; i < 1; i++)，循环条件一开始就不成立，循环一次都不执行。

程序直接跳到第18行 if (i < x)。

此时 i 是 2，x 是 1，2 < 1 不成立，程序走 else，打印出“是素数”。

数学常识：1 既不是素数也不是合数。 这段代码把 1 判成了素数，这是逻辑错误。

硬伤二：效率极低（考研会超时 TLE）

循环条件是 i < x。意思是如果输入 x = 1000003（一个很大的素数），程序要一直试到 1000002 才能停下。这在考研机试（如 PTA）绝对会报“运行超时”。

优化原理：判断素数不需要试到 x-1。如果 x 不是素数，它必然有一对因数，其中一个一定小于等于 √x。

正确写法：循环条件改为 i * i <= x。效率瞬间提升几百倍。

硬伤三：利用“副作用”做判断（可读性差，容易出 Bug）

这段代码最“巧妙”也最“危险”的地方，就是第18行的 if (i < x)。
它利用了 break 跳出后 i 不会被继续加一的特性：

如果是合数，碰到因数触发 break，此时 i 一定小于 x。

如果是素数，循环自然结束，此时 i 刚好等于 x。

27嵌套的循环

100以内的素数

如何写程序输出100以内的素数?

```c
#include <stdio.h>

int main()
{
    int x;

    //  scanf("%d", &x);
    x = 6;

    for ( x = 2; x<100; x++ )
    {
        int i;
        int isPrime = 1;    //  x是素数
        for ( i=2; i<x; i++ ) {
            if ( x % i == 0 ) {
                isPrime = 0;
                break;
            }
        }
        if ( isPrime == 1 ) {
            printf("%d ", x);
        }
    }
    printf("\n");

    return 0;
}
```

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
[Finished in 0.0s]
```

循环里面还是循环

需要就用，仅此而已

prime.c

```c

int main()
{
    int x;

    //  scanf("%d", &x);
    x = 2;
    int cnt = 0;

    // for ( x = 2; x<100; x++ )
    // while ( cnt < 50 )
    for ( x=2; cnt<50; x++ )
    {
        int i;
        int isPrime = 1;    //  x是素数
        for ( i=2; i<x; i++ ) {
            if ( x % i == 0 ) {
                isPrime = 0;
                break;
            }
        }
        if ( isPrime == 1 ) {
            printf("%d ", x);
            cnt ++;
        }
        // x++;
    }
    printf("\n");

```

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97 101 103 107 109 113 127 131 137 139 149 151 157 163 167 1
[Finished in 0.1s]
```

28跳出多种循环

凑硬币：如何用1角、2角和5角的硬币凑出10元以下的金额呢?

coin3.c coin.c

```c
#include <stdio.h>

int main()
{
    int x;
    int one, two, five;

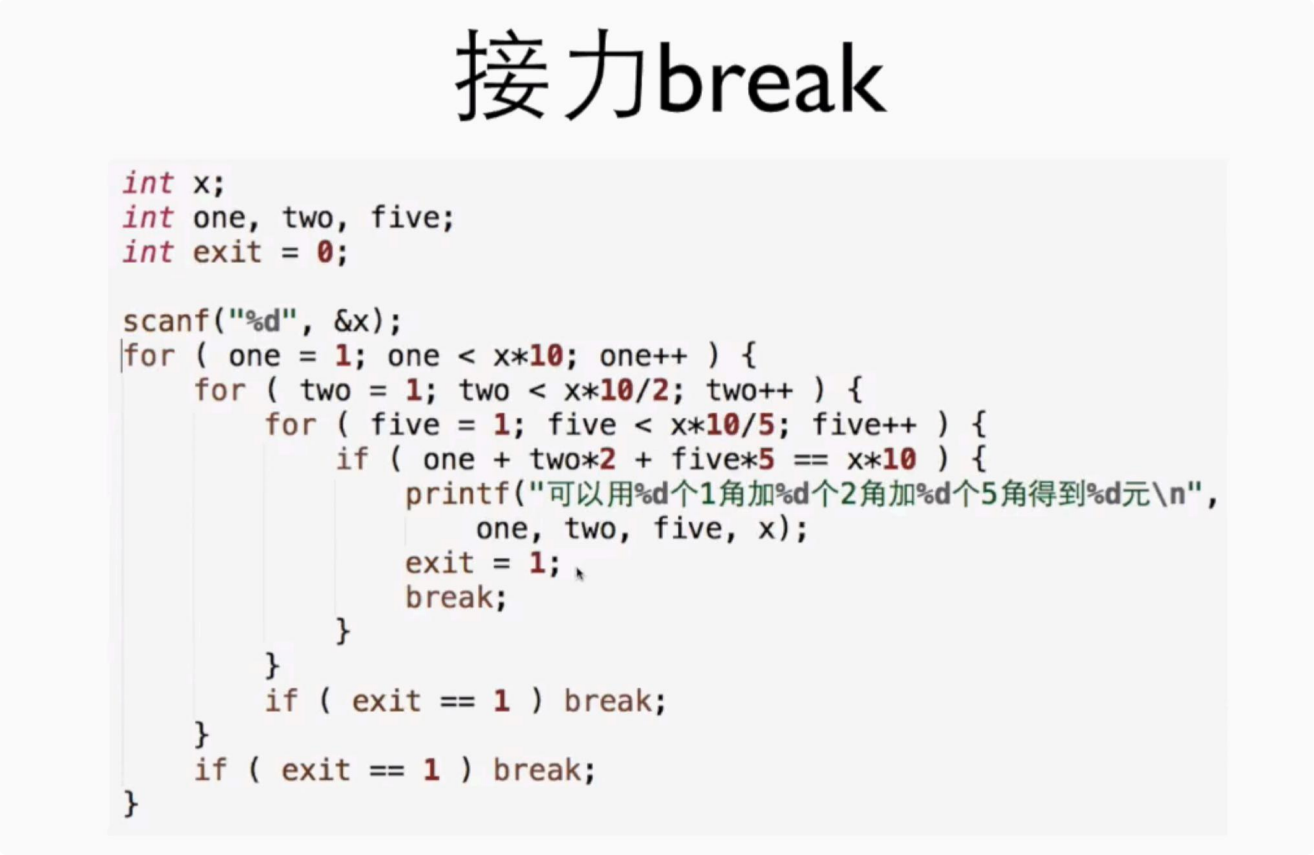
    // scanf("%d", &x);
    x = 2;
    for ( one = 1; one < x*10; one++ ) {
        for ( two = 1; two < x*10/2; two++ ) {
            for ( five = 1; five < x*10/5; five++ ) {
                if ( one + two*2 + five*5 == x*10 ) {
                    printf("可以用%d个1角加%d个2角加%d个5角得到%d元\n",
                        one, two, five, x);
                }
            }
        }
    }

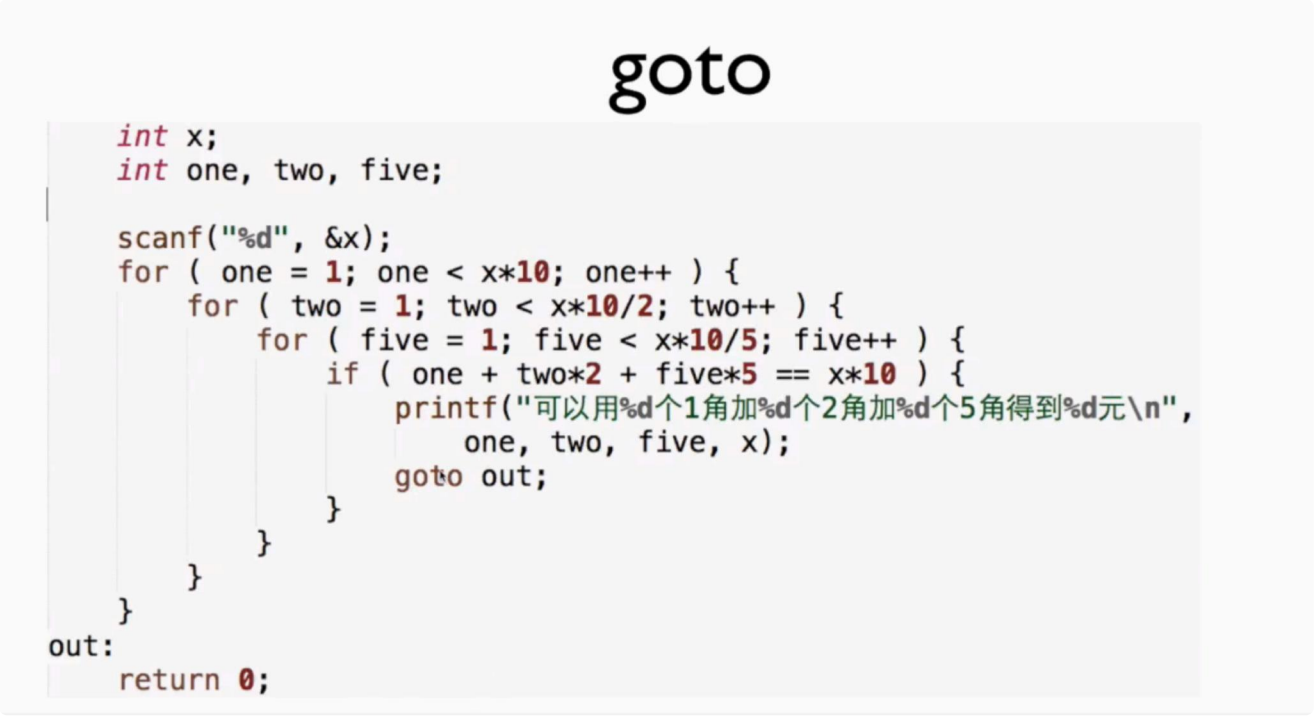
    return 0;
}
```

```
可以用1个1角加2个2角加3个5角得到2元
可以用1个1角加7个2角加1个5角得到2元
可以用2个1角加4个2角加2个5角得到2元
可以用3个1角加1个2角加3个5角得到2元
可以用3个1角加6个2角加1个5角得到2元
可以用4个1角加3个2角加2个5角得到2元
```

break和continue,只能对它所在的那层循环做

只想让它输出一种情况：





goto名声差，破坏了程序的结构性。只在有多重循环，需要从内层跳到最外面的时候使用。

29.前N项求和

1. $$f(n) = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \cdots + \frac{1}{n}$$

```c
#include <stdio.h>

int main()
{
    int n;
    int i;
    double sum = 0.0;

    scanf("%d", &n);

    for ( i = 1; i<=n; i++ ) {
        sum += 1.0/i;
    }

    printf("f(%d)=%f\n", n, sum);

    return 0;
}
```

2. $$f(n) = 1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + \cdots + \frac{1}{n}$$

```c
#include <stdio.h>

int main()
{
    int n;
    int i;
    double sum = 0.0;
    int sign = 1;

    scanf("%d", &n);
    for ( i = 1; i<=n; i++ ) {
        sum += sign*1.0/i;
        sign = -sign;
    }

    printf("f(%d)=%f\n", n, sum);

    return 0;
}
```

```c
double sum = 0.0;
// int sign = 1;
double sign = 1.0;

scanf("%d", &n);
for ( i = 1; i<=n; i++ ) {
    sum += sign/i;
    sign = -sign;
}
```

这样也可以

30.整数分解

正序分解整数

•输入一个非负整数，正序输出一个它的每一位数字

•输入：13425

•输出：1 3 4 2 5

```
 3 int main()
 4 {
 5     int x;
 6     scanf("%d", &x);
 7
 8     x = 10;
 9     int mask = 1;
10     int t = x;
11     while ( t>9 ) {
12         t /= 10;
13         mask *=10;
14     }
15     printf("x=%d, mask=%d\n", x, mask);
16     do {
17         int d = x / mask;
18         printf("%d", d);
19         if ( mask > 9 ) {
20             printf(" ");
21         }
22         x %= mask;
23         mask /= 10;
24     } while ( mask > 0 );
25     printf("\n");
26
```

```
x=10, mask=10
1 0
```

31求最大公约数

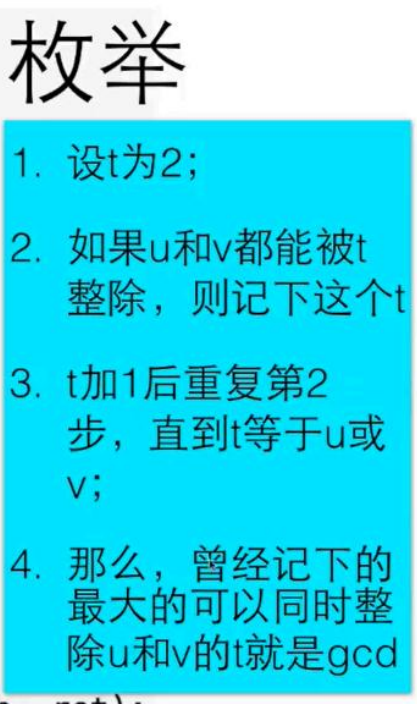
•输入两个数a和b,输出它们的最大公约数

•输入：12 18

•输出：6

```
int a,b;
int min;

scanf("%d %d", &a, &b);
if ( a<b ) {
    min = a;
} else {
    min = b;
}
int ret = 0;
int i;
for ( i = 1; i < min; i++ ) {
    if ( a%i == 0 ) {
        if ( b%i == 0 ) {
            ret = i;
        }
    }
}
printf("%d和%d的最大公约数是%d.\n", a, b, ret);
```



辗转相除法

```
/*
辗转相除法
1.如果b等于0，计算结束，a就是最大公约数;
2.否则，计算a除以b的余数，让a等于b，而b等于那个余数;
3.回到第一步。

a   b   t
12  18  12
18  12  6
12  6   0
6   0

*/
#define _CRT_SECURE_NO_WARNINGS 1
#include <stdio.h>
int main()
{
    int a, b;
    int t;
    scanf("%d %d", &a, &b);
    while (b != 0) {
        t = a % b;
        a = b;
        b = t;
```

```
            printf("a=%d,b=%d,t=%d\n", a, b, t);
        }
        printf("gcd=%d\n", a);

        return 0;
    }
```

32.数据类型、SIZEOF

1.C是有类型的语言:

•C语言的变量，必须:
•在使用前定义，并且
•确定类型

•C以后的语言向两个方向发展:
•C++/Java更强调类型，对类型的检查更严格
•JavaScript、Python、PHP不看重类型，甚至不需要事先定义

2.类型安全

•支持强类型的观点认为明确的类型有助于尽早发现程序中的简单错误

•反对强类型的观点认为过于强调类型迫使程序员面对底层、实现而非事务逻辑

•总的来说，早期语言强调类型，面向底层的语言强调类型

•C语言需要类型，但是对类型的安全检查并不足够

3.C语言的类型

> 蓝色的是C99的类型====

•整数：char、short、int、long、long long
•浮点数：float double、long double
•逻辑：bool
•指针
•自定义类型

4.类型有何不同

•类型名称: int、long、double

•输入输出时的格式化: %d、%ld、%lf

•所表达的数的范围: char<short<int<float<
double

•内存中所占据的大小: 1个字节到16个字节

•内存中的表达形式: 二进制数(补码)、编码

> 所有的类型，最终的意义是我们以什么样的方式看待它，而并不是表明，它是在内部是怎么表达的。

```c
#include <stdio.h>

int main()
{
    char c = 255;
    int i = 255;
    printf("c=%d,i=%d\n", c, i);

    return 0;
}
```

```
c=-1,i=255
```

📝 笔记：为什么 `char` 是 -1，而 `int` 是 255?

核心原理：32位截断成8位 + 符号位判定 + 补码还原

第一步：看 `int`（大房子，32位）

你写 `int i = 255;`，计算机在内存里给它分配了 **32个格子（bit）**。
255 的正数补码是：
`0000 0000 0000 0000 0000 0000 1111 1111`

- 最左边是 `0`，代表**正数。**
- 读出数值就是 **255。**

第二步：看 `char`（小房子，8位）

你写 `char c = 255;`，`char` 只有 **8个格子（1个字节）**。
因为装不下32位，C语言会发生**"强制截断"**：把左边多余的24个 `0` 全砍掉，**只留最右边的 8 位。**
截断后变成了：
`1111 1111`

第三步：为什么 `1111 1111` 是 -1？（补码的魔法）

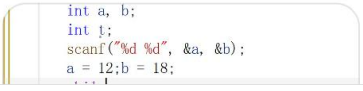
- **看最高位**：最左边的数字是 `1`，计算机一看：坏了，这是个**负数**！所以它一定是个负值。
- **取反加一（还原真值）**：既然知道是负数，就要把它的补码转成我们认识的十进制。
  - 减 1 变成：`1111 1110`
  - 按位取反变成：`0000 0001`（这就变成十进制的 1 了）
  - 加上负号：**-1**

🎯 考研必背结论（直接抄笔记）：

1. **大赋小，必截断**：把范围大的 `int` 赋值给范围小的 `char`，高位会被无情砍掉。
2. **符号位决定生死**：截断后的最高位，如果是 `1`，它就是负数。
3. **补码循环**：`char` 的范围是 `-128` 到 `127`。你硬塞 `255` 进去，超出了最大值，它就会溢出，绕着圈跑到 `-1` 的位置（就像钟表超过12点又回到1点）。

你顺着翁恺这节视频往下看，他马上就会讲 `unsigned char`（无符号字符）。如果你把代码改成 `unsigned char c = 255;`，最左边的 `1` 就不再是负号了，输出就会变回 255。这就是考研选择题最爱考的坑！

这个点非常重要，现在觉得绕很正常，多画几次二进制转换，直接拿下！



5.sizeof

•是一个运算符，给出某个类型或变量在内存中所占据的字节数

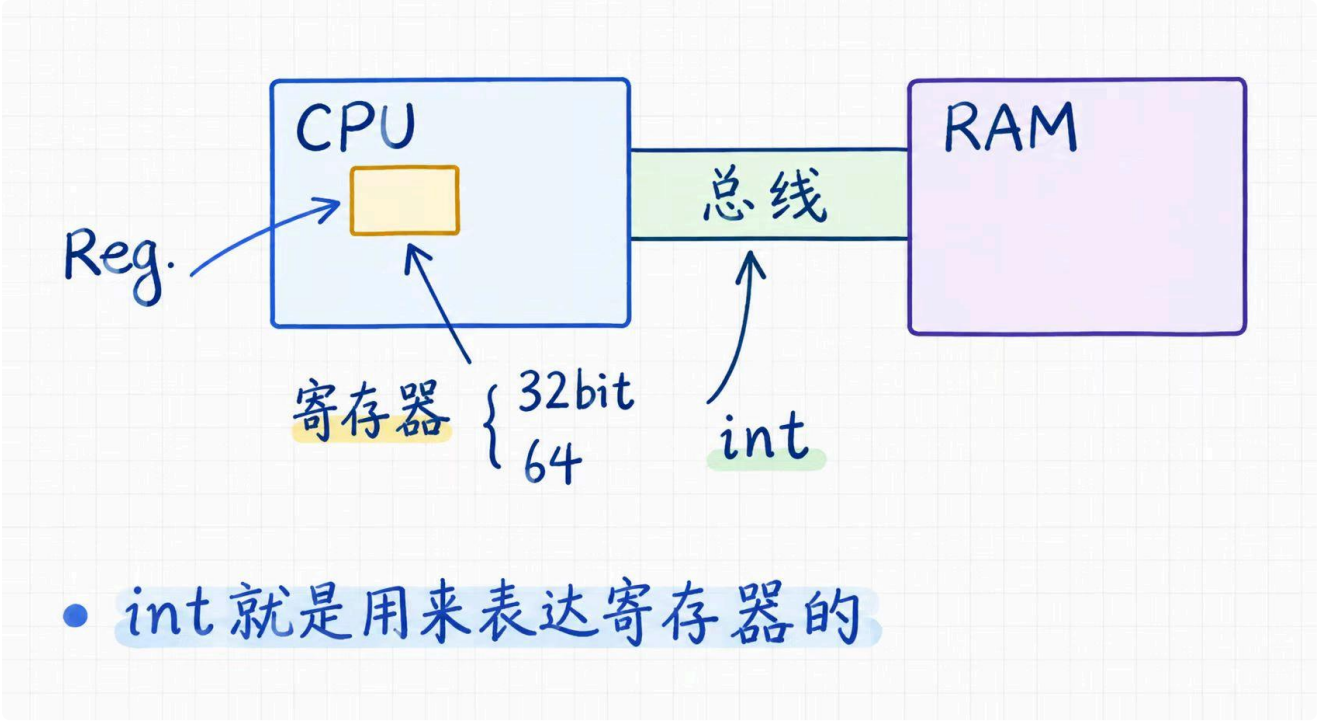
•sizeof(int)

•sizeof(i)

33除了INT的整数类型

整数

•char: 1字节(8比特)
•short: 2字节
•int: 取决于编译器(CPU)，通常的意义是“1个字”
•long: 取决于编译器(CPU)，通常的意义是“1个字”
•long long: 8字节



34.数的内部表达

1.数的内部表达

•计算机内部一切都是二进制
•18->00010010
•0->00000000
•-18—>?

2.如何表示负数:

•十进制用“-"来表示负数，在做计算的时候

3.二进制负数

•1个字节可以表达的数:
•00000000 -11111111 (0-255)
•三种方案:
1.仿照十进制，有一个特殊的标志表示负数

2.取中间的数为0，如1000000表示0，比它小的是负数，比它大的是正数
3.补码

4.补码
•考虑-1，我们希望-1+1—>0。如何能做到?
•0—>00000000
•1—>00000001
•11111111 + 00000001—>100000000

•因为0-1—>-1，所以，-1=
•(1)00000000 -00000001—>11111111
•11111111被当作纯二进制看待时，是255，被当作补码看待时是-1
•同理，对于-a，其补码就是0-a，实际是2^n-a，n是这种类型的位数

> 补码的意义就是拿补码和原码可以加出一个溢出的“零”

35.整数的范围

> 如何推算整数类型所能表达的数的范围,越界了会怎样

1.数的范围
•对于一个字节(8位)，可以表达的是:
•0000 0000-1111 1111
•其中
•0000 0000 —>0
•1111 1111 ~ 1000 0000 —> -1 ~ -128
•0000 0001~0111 1111 —>1~127

2.整数的范围
• char: 1字节: -128~127
• short: 2字节: -32768 ~32767
• int: 取决于编译器(CPU)，通常的意义是“1个字”
• long: 4字节
• longlong: 8字节

3.补码
•考虑-1，我们希望-1+1—>0。如何能做到?
•0—>0000 0000
•1—>0000 0001
•1111 1111 + 0000 0001—>1 0000 0000

•因为0-1—>-1，所以，-1=
•(1)00000000 - 00000001 —> 11111111
•11111111被当作纯二进制看待时，是255，被当作补码看待时是-1

•同理，对于-a，其补码就是0-a，实际是2^n-a，n是这种类型的位数

4.unsigned

•如果一个字面量常数想要表达自己是unsigned，可以在后面加u或U
•255U
•用l或L表示long(long)
•unsigned的初衷并非扩展数能表达的范围，而是为了做纯二进制运算，主要是为了移位

4.unsigned
•如果一个字面量常数想要表达自己是unsigned，可以在后面加u或U
•255U
•用l或L表示long(long)
•unsigned的初衷并非扩展数能表达的范围，而是为了做纯二进制运算，主要是为了移位

5.整数越界
•整数是以纯二进制方式进行计算的，所以:
•1111 1111+1—>1 0000 0000—>0



5.整数越界

•整数是以纯二进制方式进行计算的，所以:
•1111 1111+1—>1 0000 0000—>0
•0111 1111+1—> 1000 0000—>-128
•10000000-1—>01111111—> 127

36整数的格式化

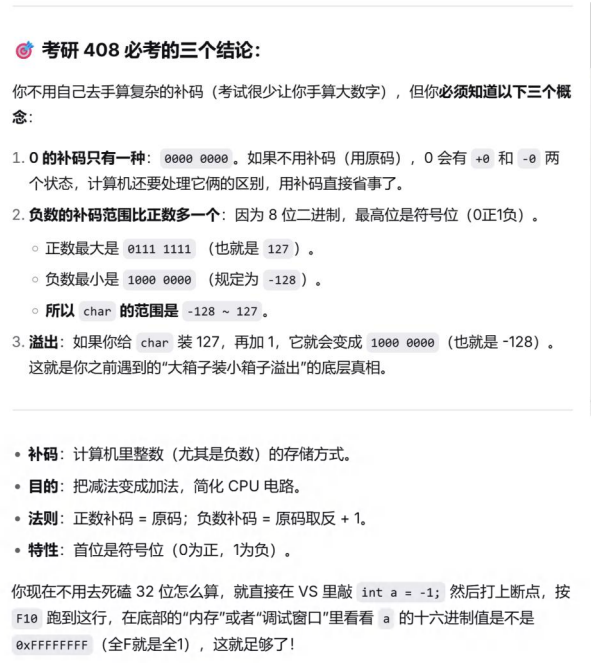
> 如何格式化输入输出整数,如何处理8进制和16进制

1.整数的输入输出

•只有两种形式: int或longlong
•%d: int
•%u: unsigned
•%ld: long long
• %lu: unsigned long long

```
1   #define _CRT_SECURE_NO_WARNINGS 1
2   #include <stdio.h>
```

```
3
4   int main()
5   {
6       char c = -1;
7       int i = -1;
8       printf("c=%u,i=%u\n", c, i);
9
10      return 0;
11  }//输出C=4294967295,i=4294967295
```



考研 408 必考的三个结论：

你不用自己去手算复杂的补码（考试很少让你手算大数字），但你**必须知道以下三个概念**：

1. **0 的补码只有一种**：`0000 0000`。如果不用补码（用原码），0 会有 `+0` 和 `-0` 两个状态，计算机还要处理它俩的区别，用补码直接省事了。
2. **负数的补码范围比正数多一个**：因为 8 位二进制，最高位是符号位（0正1负）。
   - 正数最大是 `0111 1111`（也就是 `127`）。
   - 负数最小是 `1000 0000`（规定为 `-128`）。
   - **所以 `char` 的范围是 `-128 ~ 127`。**
3. **溢出**：如果你给 `char` 装 127，再加 1，它就会变成 `1000 0000`（也就是 -128）。这就是你之前遇到的"大箱子装小箱子溢出"的底层真相。

- **补码**：计算机里整数（尤其是负数）的存储方式。
- **目的**：把减法变成加法，简化 CPU 电路。
- **法则**：正数补码 = 原码；负数补码 = 原码取反 + 1。
- **特性**：首位是符号位（0为正，1为负）。

你现在不用去死磕 32 位怎么算，就直接在 VS 里敲 `int a = -1;` 然后打上断点，按 `F10` 跑到这行，在底部的"内存"或者"调试窗口"里看看 `a` 的十六进制值是不是 `0xFFFFFFFF`（全F就是全1），这就足够了！

2.8进制和16进制

•16进制很适合表达二进制数据，因为4位二进制正好是一个16进制位
•8进制的一位数字正好表达3位二进制
•因为早期计算机的字长是12的倍数，而非8

八进制，顾名思义就是“逢 8 进 1”。它的每一位数字只能是 `0 1 2 3 4 5 6 7`。
你看二进制对应的这 8 种状态：

八进制	二进制 (3位)	八进制	二进制 (3位)
0	`000`	4	`100`
1	`001`	5	`101`
2	`010`	6	`110`
3	`011`	7	`111`

发现了吗？

- 当二进制是 `111` 时，十进制的值是 7，也就是八进制的最大值。
- 如果再大一点，加上 1，变成 `1000`（十进制的 8），因为八进制“逢 8 进 1”，所以就会进位成八进制的 `10`。

结论：3 个二进制位刚好能组合出 $2 \times 2 \times 2 = 8$ 种不同的状态，刚好塞满八进制的全部可能（0~7）。多一位就溢出了，少一位装不下。

💡 计算机为什么要用八进制和十六进制？

你想想，如果你在代码里写 `int a = 255;`，在计算机内存里其实是：
`00000000 00000000 00000000 11111111`。
人类去看这么长一串 0 和 1，眼睛直接瞎了，极度容易数错。

所以，前辈们想了个偷懒的办法：**把二进制“打包压缩”。**

- 把 3 个二进制位打成一个小包，就是**八进制**。
- 把 4 个二进制位打成一个小包，就是**十六进制**（因为 $2^4 = 16$）。

比如你之前看 `-1` 的补码，32 个 1，写成十六进制就是 `0xFFFFFFFF`（8 个 F，是不是短多了，也直观多了）。

37选择整数类型

> 没什么特殊需要就只用int就好了
> 选择整数类型

•为什么整数要有那么多种？
•为了准确表达内存，做底层程序的需要

•没有特殊需要，就选择int

•现在的CPU的字长普遍是32位或64位，一次内存读写就是一个int，一次计算也是一个int，选择更短的类型不会更快，甚至可能更慢

•--现代的编译器一般会设计内存对齐，所以更短的类型实际在内存中有可能也占据一个int的大小(虽然sizeof告诉你更小)

•unsigned与否只是输出的不同，内部计算是一样的

38浮点类型

1.浮点类型

float（字长32，有效数字7）

范围： $\pm(1.20 \times 10^{-38} \sim 3.40 \times 10^{38}),\ 0,\ \pm\inf,\ \text{nan}$

double（字长64，有效数字15）

范围： $\pm(2.2 \times 10^{-308} \sim 1.79 \times 10^{308}),\ 0,\ \pm\inf,\ \text{nan}$

2.



3.输出精度

- 在%和f之间加上.n可以指定输出小数点后几位，这样的输出是做4舍5入的
- printf("%.3f\n", -0.0049);
- printf("%.30f\n", -0.0049);
- printf("%.3f\n", -0.00049);

39浮点的范围与精度

1.超过范围的浮点数

- printf输出inf表示超过范围的浮点数:±∞
- print输出nan表示不存在的浮点数

2.浮点运算的精度

```c
#define _CRT_SECURE_NO_WARNINGS 1
#include <stdio.h>

int main()
{
    float a, b, c;

    a = 1.345f;
    b = 1.123f;
    c = a + b;
    if (c == 2.468)
        printf("相等\n");
```

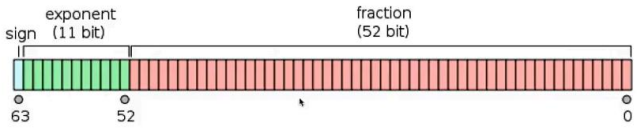
```
13      else
14          printf("不相等！  c=%.10f,或%f\n", c, c);//输出这个
15
16      return 0;
17  }
18
19  /*
20  按照人的数学，`1.345 + 1.123`绝对等于 `2.468`。但是，计算机内存里的`float`是二进制小数，它无法精准地表示`2.468`这个十进制小数，只能无限逼近，变成了`2.4679999...
21
22  永远、绝对不要用 `==` 去判断两个浮点数是否相等！如果非要判断，必须做差值，比如`if (fabs(c - 2.468) < 1e-6)`（差值小于百万分之一就算相等）
23  这是《计算机组成原理》极其经典的选择题考点。
24  */
```

- 带小数点的字面量是double而非float
- float需要用f或F后缀来表明身份

- `fI==f2`可能失败
- fabs(f1-f2)< le-12

> Android计算器低级错误?都是二进制惹的祸!
> http://www.guokr.com/article/27173/

3.浮点数的内部表达是编码模式



> 图片来源:wikipedia.org

- 浮点数在计算时是由专用的硬件部件实现的
- 计算double和float所用的部件是一样的

4.选择浮点类型
- 如果没有特殊需要，只使用double
- 现代CPU能直接对double做硬件运算，性能不会比float差，在64位的机器上，数据存储的速度也不比float慢

40字符类型

1.字符类型

- char是一种整数，也是一种特殊的类型:字符。这是因为:
- 用单引号表示的字符字面量:'a,1
- "也是一个字符:
- printf和scanf里用%c来输入输出字符

2.字符的输入输出

- 如何输入1这个字符给charc?
- scanf("%c",&c);->1
- scanf("%d", &i); c=i; ->49
- '1'的ASCI编码是49，所以当 `c==49` 时，它代表'1'
- printf("%i %cln", C,c);
- 一个49各自表述!

3.混合输入

- 有何不同?
- scanf("%d %c", &i, &c);
- scanf("%d%c", &i, &c);

4.字符计算

```c
#define _CRT_SECURE_NO_WARNINGS 1
#include <stdio.h>
int main()
{
    int i;
    char c;
    scanf("%d %c", &i, &c);
    printf("i=%d,c=%d,c='%c'\n", i, c, c);

    return 0;
}
```

输入：7 7 7
输出：i=7,c=55,c='7'

1. `%d` —— 整数（Decimal）

- **全称**：Decimal Integer（十进制整数）。
- **负责对象**：`int` 类型的变量。
- **作用**：把内存里的数据当成普通的阿拉伯数字来处理。
- **举例**：`printf("%d", 65);` 屏幕上就输出 `65`。

2. `%c` —— 字符（Character）

- **全称**：Character。
- **负责对象**：`char` 类型的变量。
- **作用**：把内存里的数据当成**ASCII 码表**里的字符来处理。
- **举例**：`printf("%c", 65);` 屏幕上不会输出 65，而是去查 ASCII 表，输出大写字母 `A`。

`'7'` 的底层数字刚好就是 **55**！

```c
#define _CRT_SECURE_NO_WARNINGS 1
#include <stdio.h>
int main()
{
    char c = 'A';
    c++;//如果是C+=2，输出C
    printf("%c\n", c);

    return 0;
}//输出B
```

- 一个字符加一个数字得到ASCII码表中那个数之后的字符
- 两个字符的减，得到它们在表中的距离

5.大小写转换

- 字母在ASCII表中是顺序排列的
- 大写字母和小写字母是分开排列的，并不在一起
- ‘a'-‘A'可以得到两段之间的距离，于是
- a+'a'-'A”可以把一个大写字母变成小写字母，而
- a+'A'-'a'可以把一个小写字母变成大写字母

41逃逸字符

1.逃逸字符

- 用来表达无法印出来的控制字符或特殊字符，它由一个反斜杠“”开头，后面跟上另一个字符，这两个字符合起来，组成了一个字符

```
printf("请分别输入身高的英尺和英寸，"“如输入\"57"表示5英尺7英寸:");
```

字符	意义	字符	意义
\b	退回一格	\"	双引号
\t	到下一个表格位	\'	单引号
\n	换行	\\	反斜杠本身
\r	回车		

制表位
- 每行的固定位置
- 一个\t使得输出从下一个制表位开始:
- 用t才能使得上下两行对齐

回车换行

- 源自打字机的动作



42类型转换

1.自动类型转换
- 当运算符的两边出现不一致的类型时，会自动转换成较大的类型
- 大的意思是能表达的数的范围更大
- char -> short-> int->long->long long
- int -> float -> double
- 对于printf，任何小于int的类型会被转换成int;float会被转换成double
- 但是scanf不会，要输入short，需要%hd

2.强制类型转换
- 要把一个量强制转换成另一个类型(通常是较小的类
- (类型)值
- 比如:
- (int)10.2
- (short)32
- 注意这时候的安全性，小的变量不总能表达大的量:
- (short)32768

> 只是从那个变量计算出了一个新的类型:的值，它并不改变那个变量，无论是值还是类型都不改变

• 强制类型转换的优先级高于四则运算

43逻辑类型

bool

• `#include <stdbool.h>`
• 之后就可以使用bool和true、false

```c
#define _CRT_SECURE_NO_WARNINGS 1
#include <stdio.h>
#include<stdbool.h>

int main()
{
    bool b = 6 > 5;
    bool t = true;
    t = 2;
    printf("%d\n", b);

    return 0;
}//输出1
```

44逻辑运算

1.逻辑运算

• 逻辑运算是对逻辑量进行的运算，结果只有0或1
• 逻辑量是关系运算或逻辑运算的结果

运算符	描述	示例	结果
!	逻辑非	！a	如果a是true结果就是false，如果a是false结果就是true
&&	逻辑与	a&&b	如果a和b都是true，结果就是true;否则就是false
||	逻辑或	a ll b	如果a和b有一个是true，结果为true;两个都是false，结果为false

2.TRY

• 如果要表达数学中的区间，如:x∈(4,6)或x∈[4,6]，应该如何写C的表达式?--> x>4 && x<6

> 像4<x<6这样的式子，不是C能正确计算的式子，因为4<x的结果是一个逻辑值(0或1)

• 如何判断一个字符c是否是大写字母?
• c>='A' && c<='Z'

3.理解一下
• age>20 && age <30
• index <0 || index>99
• !age<20

4.优先级
• !>&&>ll
• !done && (count >MAX)

优先级	运算符	结合性	
1	()	从左到右	
2	！ +-++--	从右到左（单目的+和-）	
3	*/%	从左到右	
4	+ -	从左到右	
5	<<=>>=	从左到右	
6	== !=	从左到右	
7	&&	从左到右	
8	||	从左到右	
9	= += -= *= /= %=	从右到左	短路

5.短路
• 逻辑运算是自左向右进行的，如果左边的结果已经能够决定结果了，就不会做右边的计算
• `a==6 && b==1`
• `a==6 && b+=1`
• 对于&&，左边是false时就不做右边了
• 对于||，左边是true时就不做右边了

> 不要把赋值，包括复合赋值组合进表达!

45.条件和逗号运算

条件运算
1.条件运算符

• count = (couht > 20) ? count -10 :count+10;
• 条件、条件满足时的值和条件不满足时的值

```
if(count>20) count=count-10;
else
count=count+10;
```

2.优先级

• 条件运算符的优先级高于赋值运算符，但是低于其他运算符

m<n ? x : a+5
a++ >=1 && b--> 2 ? a : b
x=3 * a > 5 ? 5 : 20

3.嵌套条件表达式

• count =(count >20)? (cout < 50)?count -10 : count - 5 : (count < 10)? count + 10 : count + 5;
• 条件运算符是自右向左结合的
• w<x?x+w:x<y?x:y

> 不建议使用嵌套的条件表达式

逗号运算

1.逗号运算符

• 逗号用来连接两个表达式，并以其右边的表达式的值作为它的结果。逗号的优先级是所有的运算符中最低的，所以它两边的表达式会先计算;

• 逗号的组合关系是自左向右，所以左边的表达式会先计算，而右边的表达式的值就留下来作为逗号运算的结果。

2.在for中使用

• for (i=0,j=10;i<j;i++,j--) .....